Computational Physics

Lecture 8: Ordinary Differential Equations (ODE)

1st-order differential equations in one variable

$$\frac{dx}{dt} = f(x, t),$$

- Euler's method
- Runge-Kutta methods
- Infinite range

Differential equations with multiple variables

2nd-order differential equations

Adaptive timestep

Other methods for differential equations

- Leapfrog method
- Verlet method

Boundary value problems

- Shooting method

References: Chapter 8 of *Computational Physics* by Mark Newman

Ordinary Differential Equations (ODE)

The most common use of computer is for solving differential equations

Ordinary differential equation of order *N*: an equation involving one function *x* of a single independent variable t, together with derivatives of *x* with respect to t, the highest of which has order N.

$$F\left(t, x(t), \frac{dx}{dt}, \frac{d^2x}{dt^2}, \ldots, \frac{d^Nx}{dt^N}\right) = 0$$

Not all derivatives need to appear. It is in an **explicit form** if it appears as following

$$\frac{d^Nx}{dt^N} = f\left(t, x(t), \frac{dx}{dt}, \frac{d^2x}{dt^2}, \ldots, \frac{d^{N-1}x}{dt^{N-1}}\right)$$

For example, a first-order one-variable ODE in explicit form is

$$\frac{dx}{dt} = f(x, t)$$

Typically you need to make it explicit to solve numerically.

Ordinary Differential Equations (ODE)

To calculate a full solutions of ODEs, we also need either:

- an initial condition at a certain t_0:

$$x(t_0) = x_0,\ \frac{dx}{dt}(t_0) = x_1, \ldots, \frac{d^{N-1}x}{dt^{N-1}}(t_0) = x_{\mathrm{N}}$$

- a boundary condition: mixed conditions that can be given at different t_k for x and its derivatives up to order $N-1$; altogether N conditions

E.g. for an equation for a tight string of the form $\frac{d^2y}{dx^2} = f(y,x)$

one would like to specify values at specific endpoints of a interval $y(a)$ and $y(b)$

Ordinary Differential Equations (ODE)

Set of ordinary differential equation of order *N*: involves a set of equations involving multiple functions **x** of a single independent variable t, $\mathbf{x} = (x_1, x_2, \dots, x_m)$

$$F_i\left(t, \mathbf{x}(t), \frac{d\mathbf{x}}{dt}, \frac{d^2\mathbf{x}}{dt^2}, \dots, \frac{d^N\mathbf{x}}{dt^N}\right) = 0 \qquad i = 1, \dots m$$

Explicit form:

$$\frac{d^N\mathbf{x}}{dt^N} = f_i\left(t, \mathbf{x}(t), \frac{d\mathbf{x}}{dt}, \frac{d^2\mathbf{x}}{dt^2}, \dots, \frac{d^{N-1}\mathbf{x}}{dt^{N-1}}\right) \qquad i = 1, \dots m$$

First-order Differential Equations with one variable

Let's start with a first-order ODE: $\frac{dx}{dt} = f(x,t)$

with an initial condition $x(t_0) = x_0$

Cases you studied involved the possibility of finding a solution e.g. linear equations, or cases where you can perform variable separation:

$$\frac{dx}{dt} = \frac{2x}{t}$$

Suppose instead we have an equation no longer separable and nonlinear (we have either a power or a non-linear function in *x*):

$$\frac{dx}{dt} = \frac{2x}{t} + \frac{3x^2}{t^3}$$

much more difficult to obtain an analytic solution. Let's see what type of numerical methods can be adopted

Euler's method

Given $\frac{dx}{dt} = f(x,t)$ and an initial conditions $x(t)$ that fixes *x* at some time *t*

We now can write *x* at some later time *t+h* using Taylor expansion: $x(t+h) = x(t) + h\frac{dx}{dt} + O(h^2)$

Given that $dx/dt = f(x,t)$ and neglecting the high-order terms $O(h^2)$ we get a practical formula for the evolution at *t+h:*

$$x(t+h) = x(t) + hf(x,t) + O(h^2)$$

Euler method

We can iteratively apply this relation starting from *t = 0* to evaluate *x(t)* at *t > 0*.
We don't get the value at all *t*, but only at *h* intervals. With *h* sufficiently small we get a good picture of *x(t)*

This is the simplest method for solving ODEs numerically.

Euler's method

The error in the approximation is $O(h^2)$ because we neglected $\frac{1}{2}h^2\frac{d^2x}{dt^2}$ and higher orders.
Thus, the more *h* gets smaller the better the error is.

But suppose we are integrating between t $\in [a, b]$, if $N = \frac{b-a}{h}$ is the number of steps
the **total cumulative error** is one order higher than the step error:

$$O(h^2)N = \mathrm{O}(h^2)\frac{b-a}{h} = O(h)$$

Halving *h* means I am halving the total error. Not convenient to decrease h, considering for each halving we are doubling the operations

Better methods are available that decrease error order while not increasing operations order

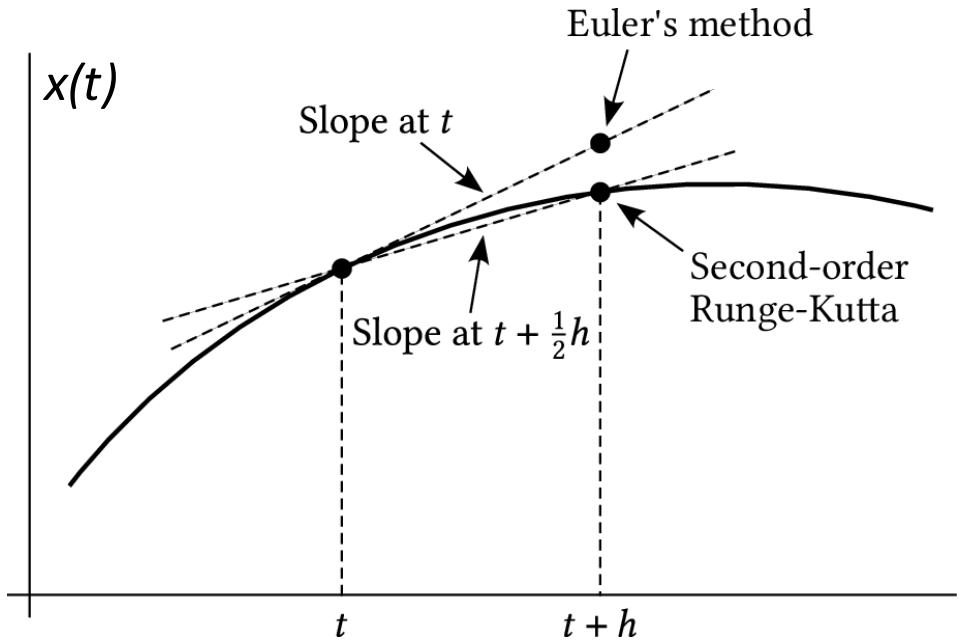
Runge-Kutta method (2nd order)

You might think to improve Euler's method we would need the second derivative of *f*, either analytically or numerically:

$$\frac{1}{2}h^2\frac{d^2x}{dt^2} = \frac{1}{2}h^2\frac{df}{dt}$$

The Runge-Kutta is a set method which gives high accuracy without computing derivatives

Let's start **with second-order Runge-Kutta** (also known as midpoint method)



A graphical representation of Euler's method is to consider the real x(t) curve. $dx/dt = f(x,t)$ tells the value at *x* of the slope.

Extrapolating the slope to *t+h* gives actually the point estimated in Euler's method: $x(t+h) = x(t) + hf(x,t)$
If *x(t)* is curved the estimate is only approximate.

If instead we use the slope at the midpoint $t + \frac{1}{2}h$ we would get a better approximation

Runge-Kutta method (2nd order)

Start from Taylor expansion around $t + \frac{1}{2}h$ to get value in *t+h* and in *t*

$$x(t+h) = x\left(t+\tfrac{1}{2}h\right) + \tfrac{1}{2}h\left(\frac{\mathrm{d}x}{\mathrm{d}t}\right)_{t+\frac{1}{2}h} + \tfrac{1}{8}h^2\left(\frac{\mathrm{d}^2x}{\mathrm{d}t^2}\right)_{t+\frac{1}{2}h} + O(h^3)$$

$$x(t) = x\left(t+\tfrac{1}{2}h\right) - \tfrac{1}{2}h\left(\frac{\mathrm{d}x}{\mathrm{d}t}\right)_{t+\frac{1}{2}h} + \tfrac{1}{8}h^2\left(\frac{\mathrm{d}^2x}{\mathrm{d}t^2}\right)_{t+\frac{1}{2}h} + O(h^3)$$

Subtracting the second eq. from the first there is a miraculous cancellation of h^2 terms:

$$x(t+h) = x(t) + h\left(\frac{\mathrm{d}x}{\mathrm{d}t}\right)_{t+\frac{1}{2}h} + O(h^3) \;= x(t) + h\,f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right) + O(h^3)$$

Only problem is we need value of $\mathrm{x}(t+\frac{1}{2}h)$. We can use Euler's method for estimate this value

$$x\left(t+\tfrac{1}{2}h\right) = x(t) + \tfrac{1}{2}h\,f(x,t)$$

Runge-Kutta method (2nd order)

The **second-order Runge-Kutta** looks like this:

$$
\begin{aligned}
k_1 &= h\, f(x, t), \\
k_2 &= h\, f\left(x + \tfrac{1}{2}k_1,\, t + \tfrac{1}{2}h\right), \\
x(t+h) &= x(t) + k_2.
\end{aligned}
$$

What is the step error?

We neglected $O(h^3)$ terms when writing the second formula, but are we introducing lower order errors starting from Euler's step in the first formula?

Runge-Kutta method (2nd order)

Take $f\left(x(t)+\frac{1}{2}k_1,\, t+\frac{1}{2}h\right)$

and see how the error in Euler's eq. contribute inside its expansion in its first argument around $x\left(t+\frac{1}{2}h\right)$:

$$f\left(x(t)+\tfrac{1}{2}k_1,\, t+\tfrac{1}{2}h\right) = f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right) + \underbrace{\left[x(t)+\tfrac{1}{2}k_1 - x(t+\tfrac{1}{2}h)\right]}_{O(h^2)}\left(\frac{\partial f}{\partial x}\right)_{x(t+\frac{1}{2}h),\, t+\frac{1}{2}h} + \underbrace{O\left(\left[x(t)+\tfrac{1}{2}k_1 - x(t+\tfrac{1}{2}h)\right]^2\right)}_{O(h^2)}$$

Note that from Euler's eq. the 2nd and 3rd terms are $O(h^2)$:

$$x\left(t+\tfrac{1}{2}h\right) = x(t)+\tfrac{1}{2}h\,f(x,t)+O(h^2) = x(t)+\tfrac{1}{2}k_1+O(h^2) \longrightarrow x(t)+\tfrac{1}{2}k_1 - x(t+\tfrac{1}{2}h) = O(h^2)$$

We remain with:

$$f\left(x(t)+\tfrac{1}{2}k_1,\, t+\tfrac{1}{2}h\right) = f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right) + O(h^2)$$

Hence the second Eq. in RK has the same error as the last formula and does not introduce a lower-order one

$$k_2 = h\,f\left(x(t+\tfrac{1}{2}h), t+\tfrac{1}{2}h\right) + O(h^3)$$

Runge-Kutta method (2nd order)

Euler is a first-order method:
step error is order h^2
step accuracy is order h
global error is order h
global accuracy doesn't scale with h

Second-order RK method is a second-order method:
step error is order h^3
step accuracy is order h^2
global error is order h^2
global accuracy is order h

The 2-nd order RK algorithm increase a bit the number of operations but enhances the accuracy by one order of magnitude!

Runge-Kutta method (4th order)

The above logic can be generalized: perform Taylor expansion of *f(x,t)* at various points, take the right linear combination and you are able to cancel higher-order terms in h^3, h^4, etc

The equations however become more complex. The 4th-order Runge-Kutta method is often considered a sweet spot.

$$
\begin{aligned}
k_1 &= h\, f(x, t), \\
k_2 &= h\, f\left(x + \tfrac{1}{2}k_1,\, t + \tfrac{1}{2}h\right), \\
k_3 &= h\, f\left(x + \tfrac{1}{2}k_2,\, t + \tfrac{1}{2}h\right), \\
k_4 &= h\, f(x + k_3,\, t + h), \\
x(t+h) &= x(t) + \tfrac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4).
\end{aligned}
$$

Step accuracy of order h^4 (global accuracy is order h^3)

The classical 4th-order Runge-Kutta method is a good first choice for solving physics ODEs.

1st-order ODE: example

Take the following differential equation and solve it using Euler, 2nd and 4th order Runge Kutta methods

$$\frac{\mathrm{d}x}{\mathrm{d}t} = -x^3 + \sin t$$

initial condition x=0 at t=0
Integrate from t=0 to t=10

For every method, use a timestep of size h=1,0.5,0.2,0.1,0.01 and compare the results

```python
import numpy as np
import matplotlib.pyplot as plt

def f(x, t):
    return -x**3 + np.sin(t)

a = 0.0
b = 10.0
x0 = 0.0
hs = [1.0, 0.5, 0.2, 0.1, 0.01]

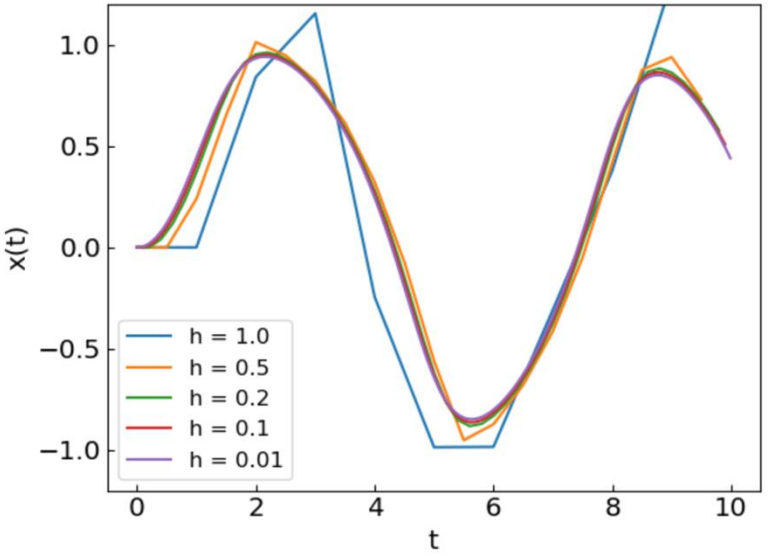
for h in hs:
    N = int((b - a) / h)
    tpoints = np.arange(a, b, h)
    x = x0
    xpoints = []
    for t in tpoints:
        xpoints.append(x)
        x += h * f(x, t)
    plt.plot(tpoints, xpoints, label=f"h = {h}")

plt.xlabel("t")
plt.ylabel("x(t)")
plt.legend()
plt.show()
```

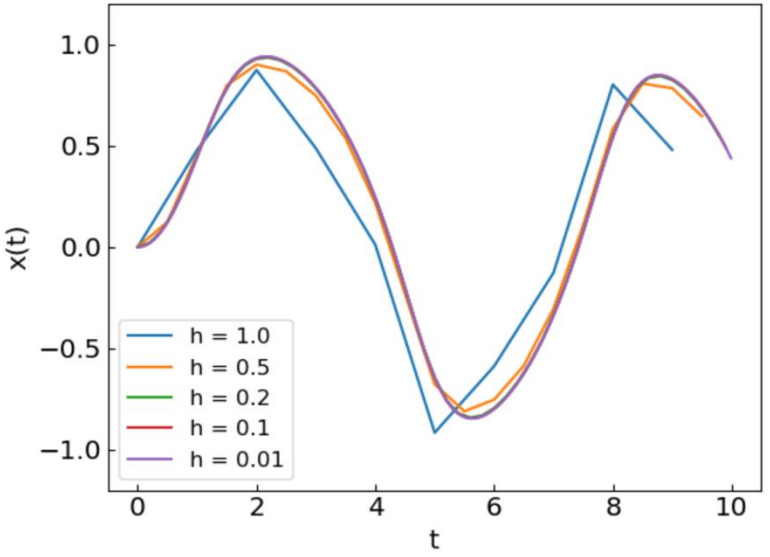
need to change this line for other methods

1st-order ODE: example

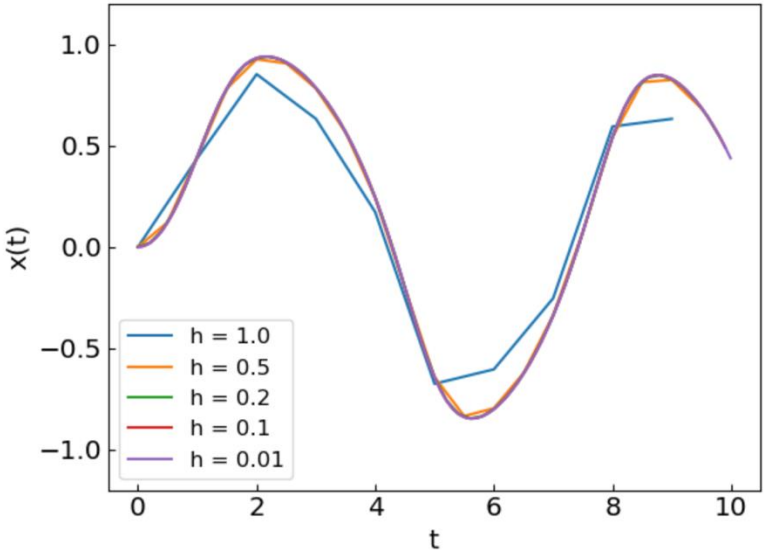
Euler



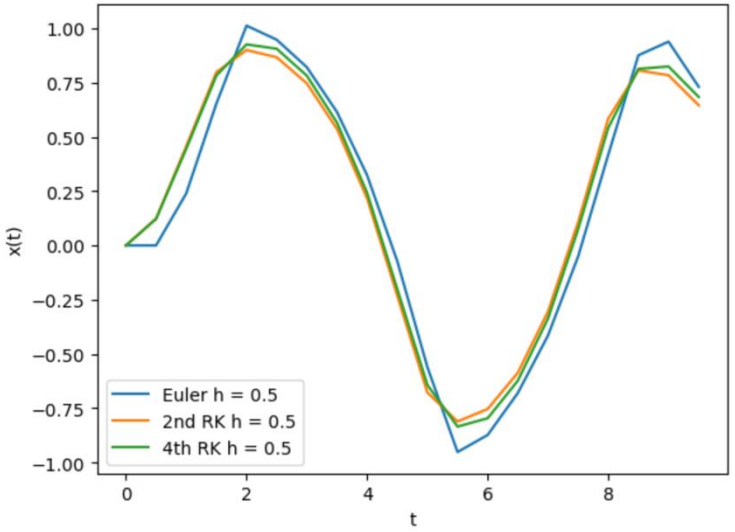
2nd order Runge-Kutta



4th order Runge-Kutta



Comparison at fixed h of different methods



1st-order ODE solution over infinite range

What if we want the solution not only up to a finite t, but for $t = \infty$? Can perform change of variable first:

$$u = \frac{t}{1+t} \qquad \text{or} \qquad t = \frac{u}{1-u} \qquad \text{so that when } t \to \infty \text{ we have } u \to 1$$

Using the chain rule we can rewrite

$$\frac{dx}{dt} = f(x,t) \longrightarrow \frac{\mathrm{d}x}{\mathrm{d}u}\frac{\mathrm{d}u}{\mathrm{d}t} = f(x,t) \longrightarrow \frac{\mathrm{d}x}{\mathrm{d}u} = \underbrace{\frac{\mathrm{d}t}{\mathrm{d}u}}_{\frac{\mathrm{d}t}{\mathrm{d}u} = \frac{1}{(1-u)^2}} f\left(x, \frac{u}{1-u}\right) \longrightarrow \frac{\mathrm{d}x}{\mathrm{d}u} = (1-u)^{-2} f\left(x, \frac{u}{1-u}\right)$$

We get a new equation which we need to solve for u in between [0,1]

$$\frac{\mathrm{d}x}{\mathrm{d}u} = g(x,u) \qquad \text{with} \qquad g(x,u) = (1-u)^{-2} f\left(x, \frac{u}{1-u}\right)$$

1st-order ODE solution over infinite range: example

Use now 4th order Runge-Kutta plus change in variable to solve $$\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{1}{x^2+t^2}$$

the equation should become $$\frac{\mathrm{d}x}{\mathrm{d}u} = \frac{1}{x^2(1-u)^2+u^2}$$

solve from u=0 to u=1 with initial condition x=1 at u=0
Plot the result as a function of t

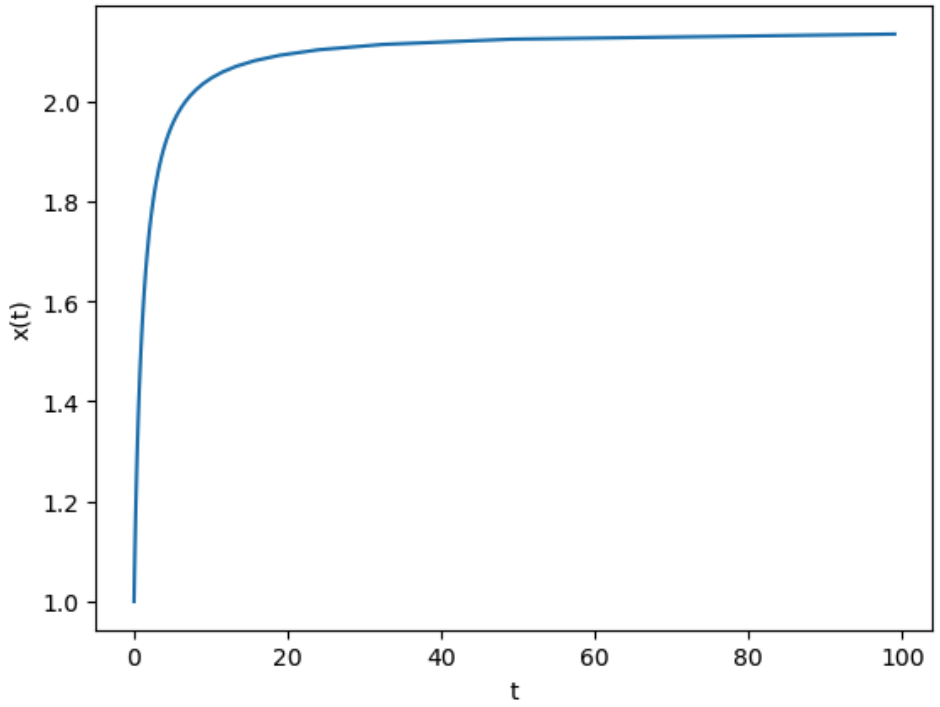
```
import numpy as np
import matplotlib.pyplot as plt

def g(x,u):
    return 1/(x**2*(1-u)**2+u**2)

a = 0.0
b = 1.0
h = 0.01

upoints = np.arange(a,b,h)
tpoints = []
xpoints = []

x = 1.0
for u in upoints:
    tpoints.append(u/(1-u))
    xpoints.append(x)
    k1 = h*g(x,u)
    k2 = h*g(x+0.5*k1,u+0.5*h)
    k3 = h*g(x+0.5*k2,u+0.5*h)
    k4 = h*g(x+k3,u+h)
    x += (k1+2*k2+2*k3+k4)/6
plt.plot(tpoints,xpoints)
#plt.xlim(0,100)
```



Systems of 1st-order ODEs

Consider now a system of *N* simultaneous first-order ODE: $x_1, x_2, \ldots, x_N$ variables, all dependent only on one independent variable *t*.

$$\frac{dx_1}{dt} = f_1(x_1, \ldots, x_N, t),$$
$$\frac{dx_2}{dt} = f_2(x_1, \ldots, x_N, t),$$
$$\ldots$$
$$\frac{dx_N}{dt} = f_N(x_1, \ldots, x_N, t).$$

In vector notation: $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t).$ with

$$\mathbf{x} = (x_1, x_2, \ldots, x_N)$$
$$f(\mathbf{x}, t) = (f_1(x_1), f_2(x_2), \ldots, f_N(x_N))$$

Very hard to solve analytically, but:

- all the methods we covered have the same structure when applied for systems of ODEs
- simply apply component by component

Systems of 1st-order ODEs

Take Taylor expansion:

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h\frac{d\mathbf{x}}{dt} + O(h^2) = \mathbf{x}(t) + h\mathbf{f}[\mathbf{x}(t), t] + O(h^2)$$

Dropping $O(h^2)$ we immediately get Euler's method for multiple variables:

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h\mathbf{f}[\mathbf{x}(t), t]$$

Same reasoning can be straightly applied to derivation of Runge-Kutta methods

$$\begin{aligned}
\boldsymbol{k_1} &= h\,\mathbf{f}[\mathbf{x}(t), t] \\
\boldsymbol{k_2} &= h\,\mathbf{f}[\mathbf{x}(t) + \mathbf{k}_1/2, t + h/2] \\
\boldsymbol{k_3} &= h\,\mathbf{f}[\mathbf{x}(t) + \mathbf{k}_2/2, t + h/2] \\
\boldsymbol{k_4} &= h\,\mathbf{f}[\mathbf{x}(t) + \mathbf{k}_3, t + h] \\
\mathbf{x}(t+h) &= \mathbf{x}(t) + \frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)
\end{aligned}$$

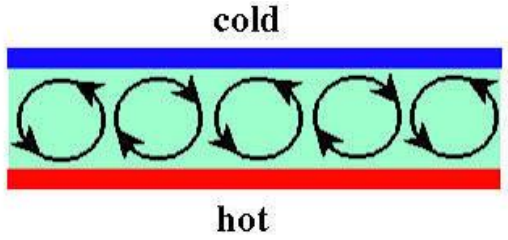
In Python these equations are easily implemented using arrays and arrays arithmetic

Systems of 1st-order ODEs: Lorenz equations

Consider set of 3 first-order ODE, also known as Lorenz equations:

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \sigma(y - x) \qquad \frac{\mathrm{d}y}{\mathrm{d}t} = \rho x - y - xz \qquad \frac{\mathrm{d}z}{\mathrm{d}t} = xy - \beta z$$

with σ, ρ, β constants



These equations were first studied by Edward Lorenz in 1963, and model a two-dimensional fluid cell which is warmed from below and cooled from above, with a resulting convective motion.

They are famous as the first example of deterministic chaos: occurrence of apparent random motion even though there is no randomness built into the equations.

Meaning of the variables

x is proportional to the intensity of convective motion (sign relates to direction of motion),
y is proportional to the temperature difference between ascending and descending currents,
z is proportional to the distortion from linearity of the vertical temperature profile.

Systems of 1st-order ODEs: Lorenz equations

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \sigma(y - x) \qquad \frac{\mathrm{d}y}{\mathrm{d}t} = \rho x - y - xz \qquad \frac{\mathrm{d}z}{\mathrm{d}t} = xy - \beta z$$

1) Write a program to integrate such equations using the 4th order Runge-Kutta algorithm.
   Use $\sigma=10$, $\rho=28$, $\beta=8/3$ and integrate from *t=0* to *t=50* with initial conditions *(x,y,z)=(0,1,0)*

2) Plot y as a function of time. Note how the motion is unpredictable

3) Plot z as a function of x

4) Sensitivity to initial conditions: start with *(x,y,z)=(0,1.0001,0) and look at the trajectory*

Systems of 1st-order ODEs: Lorenz equations

```python
import numpy as np
import matplotlib.pyplot as plt

#Lorenz parameters
sigma = 10.0
rho = 28.0
beta = 8.0 / 3.0

def f(r, t):
    x, y, z = r
    fx = sigma * (y - x)
    fy = rho * x - y - x * z
    fz = x * y - beta * z
    return np.array([fx, fy, fz], float)

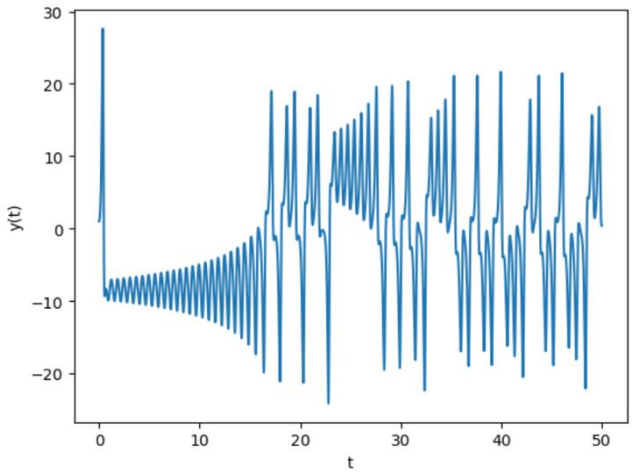
a, b = 0.0, 50.0
h = 0.01
tpoints = np.arange(a, b, h)

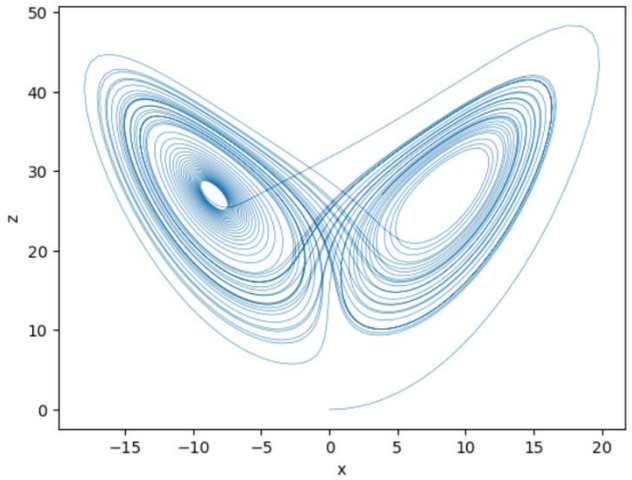
xpoints, ypoints, zpoints = [], [], []
r = np.array([0.0, 1.0, 0.0], float)

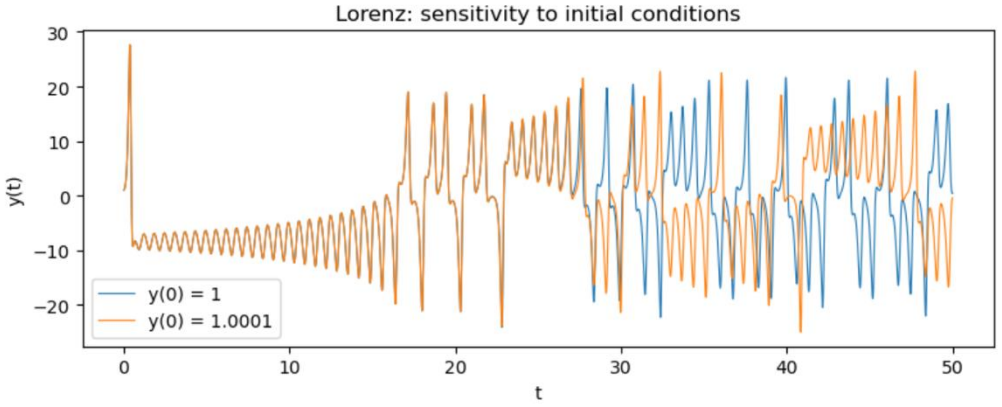
for t in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    zpoints.append(r[2])
    k1 = h * f(r, t)
    k2 = h * f(r + 0.5 * k1, t + 0.5 * h)
    k3 = h * f(r + 0.5 * k2, t + 0.5 * h)
    k4 = h * f(r + k3, t + h)
    r += (k1 + 2 * k2 + 2 * k3 + k4) / 6

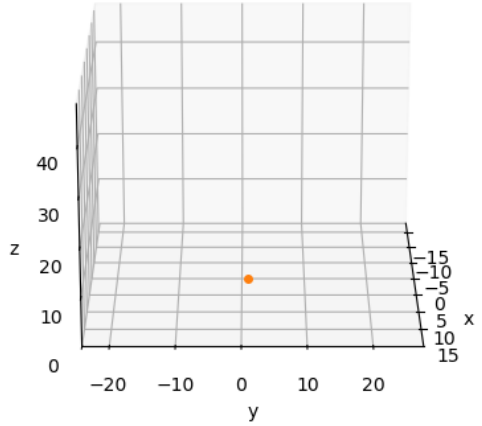
plt.plot(tpoints, ypoints)
plt.xlabel("t"); plt.ylabel("y(t)")
plt.show()

plt.plot(xpoints, zpoints, lw=0.4)
plt.xlabel("x"); plt.ylabel("z")
plt.show()
```









2nd-order and higher-order ODEs

Most of physics ODE are 2nd-order (e.g. Newton equations of motion)

A 2nd-order ODE with one dependent variable *x* has the form

$$\frac{\mathrm{d}^2x}{\mathrm{d}t^2} = f\left(x, \frac{\mathrm{d}x}{\mathrm{d}t}, t\right)$$

so now we can have a non-linear equation dependent on *x, t*, and also *dx/dt*, e.g.

$$\frac{\mathrm{d}^2x}{\mathrm{d}t^2} = \frac{1}{x}\left(\frac{\mathrm{d}x}{\mathrm{d}t}\right)^2 + 2\frac{\mathrm{d}x}{\mathrm{d}t} - x^3e^{-4t}$$

How to solve? The trick is to write it as a system of 1nd-order ODEs. For a 2nd-order ODE simply define y =dx/dt and obtain the system:

$$\begin{cases} \dfrac{\mathrm{d}x}{\mathrm{d}t} = y \\ \dfrac{\mathrm{d}y}{\mathrm{d}t} = f(x, y, t) \end{cases}$$

2nd-order and higher-order ODEs

The approach can be extended to solve higher-order ODEs:

$$\frac{\mathrm{d}^3x}{\mathrm{d}t^3} = f\left(x, \frac{\mathrm{d}x}{\mathrm{d}t}, \frac{\mathrm{d}^2x}{\mathrm{d}t^2}, t\right)$$

$$\begin{cases} \dfrac{\mathrm{d}x}{\mathrm{d}t} = y \\ \dfrac{\mathrm{d}y}{\mathrm{d}t} = z \\ \dfrac{\mathrm{d}z}{\mathrm{d}t} = f(x, y, z, t) \end{cases}$$

And to ODEs in more variables:

$$\frac{\mathrm{d}^2\mathbf{x}}{\mathrm{d}t^2} = \mathbf{f}\left(\mathbf{x}, \frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t}, t\right)$$

$$\begin{cases} \dfrac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{y} \\ \dfrac{\mathrm{d}\mathbf{y}}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}, \mathbf{y}, t) \end{cases}$$

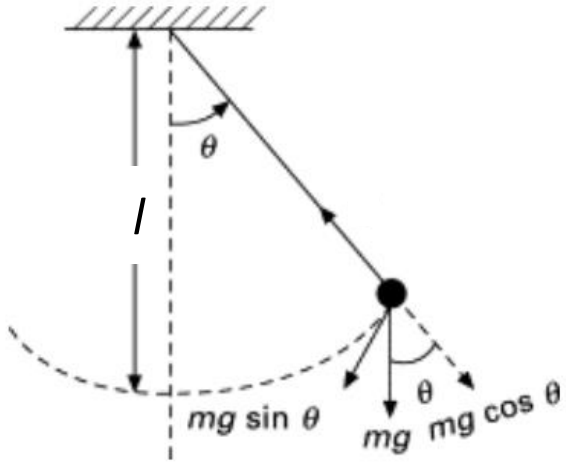
2nd-order ODE: nonlinear pendulum

A classic problem is the equation of a pendulum, which is typically is approximated as linear, but in principle is non-linear.

$$m\ell\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -mg\sin\theta$$

acceleration on tangential direction

gravitational force component on tangential direction



The 2nd-order ODE can be decomposed as

$$\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -\frac{g}{\ell}\sin\theta$$

$$\begin{cases} \dfrac{\mathrm{d}\theta}{\mathrm{d}t} = \omega \\ \dfrac{\mathrm{d}\omega}{\mathrm{d}t} = -\dfrac{g}{\ell}\sin\theta \end{cases}$$

2nd-order ODE: nonlinear pendulum

1) Use Runge-Kutta to integrate the two first-order equations, with l=10cm, g=9.81ms^{-2}

2) Calculate the angle θ as a function of time for several periods starting from a standstill at θ =179° and compare to the case of a linear approximated force: $\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -\frac{g}{\ell}\theta$

   You can use as reference for the period the one for the linear pendulum $\omega_0 = \sqrt{g/l}$.

3) Now add a small horizontal force which is oscillatory in time: $\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -\frac{g}{\ell}\sin\theta + C\cos\theta\sin\Omega t$

   Solve with C=2s^{-2} and Ω=5s^{-1}, and plot θ as a function of time from t=0s to t=100s starting at rest from θ=0° and dθ/dt=0

4) While keeping fixed C, change Ω in between [1,20] and find values for which the pendulum resonates with the driving force. E.g., you can measure the maximum excursion $A = \frac{1}{2}(\theta_{max} - \theta_{min})$ as a function of Ω

2nd-order ODE: nonlinear pendulum

```python
# ---------------------------------------------------------
# RK4 integrator
# ---------------------------------------------------------
def rk4_step(f, t, x, h):
    """One classical RK4 step.
    f : callable (t, x) -> dx/dt
    t : current time [s]
    x : state [theta, omega]
    h : step size [s]
    Returns the state at t+h.
    """
    k1 = f(t, x)
    k2 = f(t + 0.5 * h, x + 0.5 * h * k1)
    k3 = f(t + 0.5 * h, x + 0.5 * h * k2)
    k4 = f(t + h, x + h * k3)
    return x + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)

def rk4_integrate(f, t0, x0, t_end, h):
    """Integrate from t0 to t_end with fixed step h.
    f     : RHS  dx/dt = f(t, x)
    t0    : initial time [s]
    x0    : initial state [theta0, omega0]
    t_end : final time [s]
    h     : step size [s]
    Returns
      t : time array, shape (n+1,)
      x : state array, shape (n+1, 2)
          x[:, 0] = theta(t),  x[:, 1] = omega(t)
    """
    n = int(np.round((t_end - t0) / h))
    t = t0 + np.arange(n + 1) * h
    x = np.empty((n + 1, len(x0)))
    x[0] = x0
    for i in range(n):
        x[i + 1] = rk4_step(f, t[i], x[i], h)
    return t, x
```

```python
# ---------------------------------------------------------------------------
# Right-hand side functions
# ---------------------------------------------------------------------------
def f_nonlinear(t, x):
    theta, omega = x
    return np.array([omega, -(G / L) * np.sin(theta)])

def f_linear(t, x):
    theta, omega = x
    return np.array([omega, -(G / L) * theta])
```

rk4_step gives the single step evolution
rk4_integrate, gives all the trajectory. This is a good idea only with small and handleable trajectory dataset.

For very long trajectories, just define the rk4_step and use it in the main.

2nd-order ODE: nonlinear pendulum

```
#Define global quantities
L = 0.10          # m  (10 cm)
G = 9.81          # m s^-2
OMEGA0 = np.sqrt(G / L)  # natural small-oscillation frequency [rad/s]
print(f"ℓ = {L} m,  g = {G} m/s²,  ω0 = sqrt(g/ℓ) = {OMEGA0:.6f} rad/s")
print(f"T0 = 2π/ω0 = {2 * np.pi / OMEGA0:.6f} s")
```

```
ℓ = 0.1 m,  g = 9.81 m/s²,  ω0 = sqrt(g/ℓ) = 9.904544 rad/s
T0 = 2π/ω0 = 0.634374 s
```

```
# Part 1–2. Free pendulum: nonlinear vs linear, start near the unstable top
#initial conditions
theta0_deg = 179.0
x0 = np.array([np.deg2rad(theta0_deg), 0.0])

#simulate for several periods of linear oscillator
t_end = 8 * T0
h = 1e-3

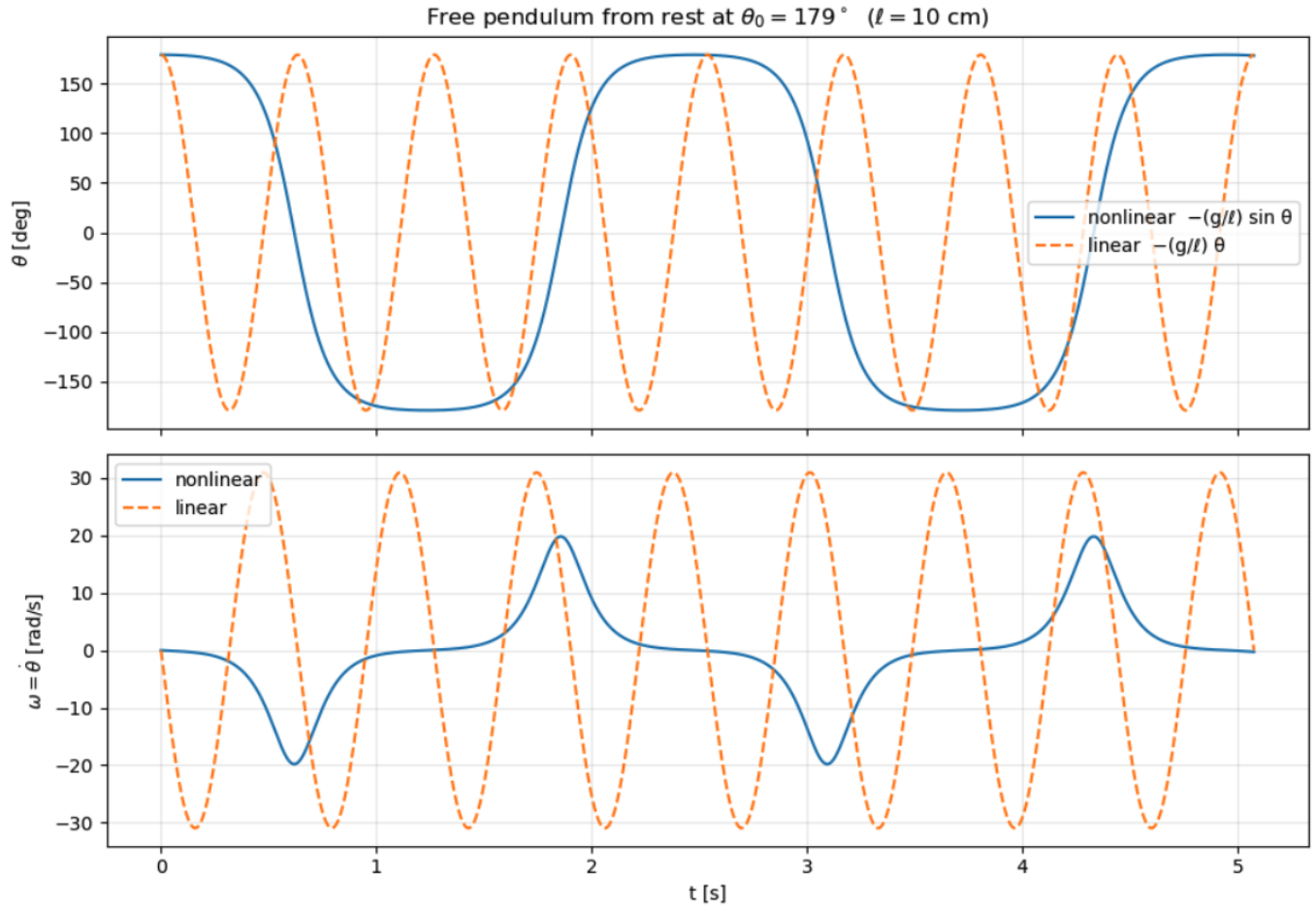
#compute trajectories for linear and non-linear oscillator
t_nl, x_nl = rk4_integrate(f_nonlinear, 0.0, x0, t_end, h)
t_lin, x_lin = rk4_integrate(f_linear, 0.0, x0, t_end, h)
```

```
fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
axes[0].plot(t_nl, np.rad2deg(x_nl[:, 0]), label="nonlinear  -(g/ℓ) sin θ")
axes[0].plot(t_lin, np.rad2deg(x_lin[:, 0]), "--", label="linear  -(g/ℓ) θ")
axes[0].set_ylabel(r"$\theta$ [deg]")
axes[0].set_title(
    rf"Free pendulum from rest at $\theta_0={theta0_deg:.0f}^\circ$  "
    rf"($\ell=10$ cm)"
)
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(t_nl, x_nl[:, 1], label="nonlinear")
axes[1].plot(t_lin, x_lin[:, 1], "--", label="linear")
axes[1].set_xlabel("t [s]")
axes[1].set_ylabel(r"$\omega = \dot\theta$ [rad/s]")
axes[1].legend()
axes[1].grid(True, alpha=0.3)
plt.show()
```

2nd-order ODE: nonlinear pendulum

The linear approximation is not working for large initial angles



2nd-order ODE: nonlinear pendulum

```
# Part 3. Driven pendulum

def make_f_driven(C, Omega):
    def f(t, y):
        theta, omega = y
        return np.array(
            [omega, -(G / L) * np.sin(theta) + C * np.cos(theta) * np.sin(Omega * t)]
        )
    return f

C = 2.0
Omega = 5.0
t_end = T0*100
h = 1e-3
x0 = np.array([0.0, 0.0])

t_d, x_d = rk4_integrate(make_f_driven(C, Omega), 0.0, x0, t_end, h)

fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(t_d, np.rad2deg(x_d[:, 0]), lw=0.8)
ax.set_xlabel("t [s]")
ax.set_ylabel(r"$\theta$ [deg]")
ax.set_title(
    rf"Driven pendulum  C={C},  Ω={Omega_drive} s$^{{-1}}$,  "
    r"$\theta(0)=\dot\theta(0)=0$"
)
ax.grid(True, alpha=0.3)
fig.tight_layout()
plt.show()
```

2nd-order ODE: nonlinear pendulum

```
# Part 4. Resonance scan in Ω (C fixed)
Omegas = np.linspace(1.0, 20.0, 40)
# also include ω0 explicitly
Omegas = np.unique(np.sort(np.append(Omegas, OMEGA0)))

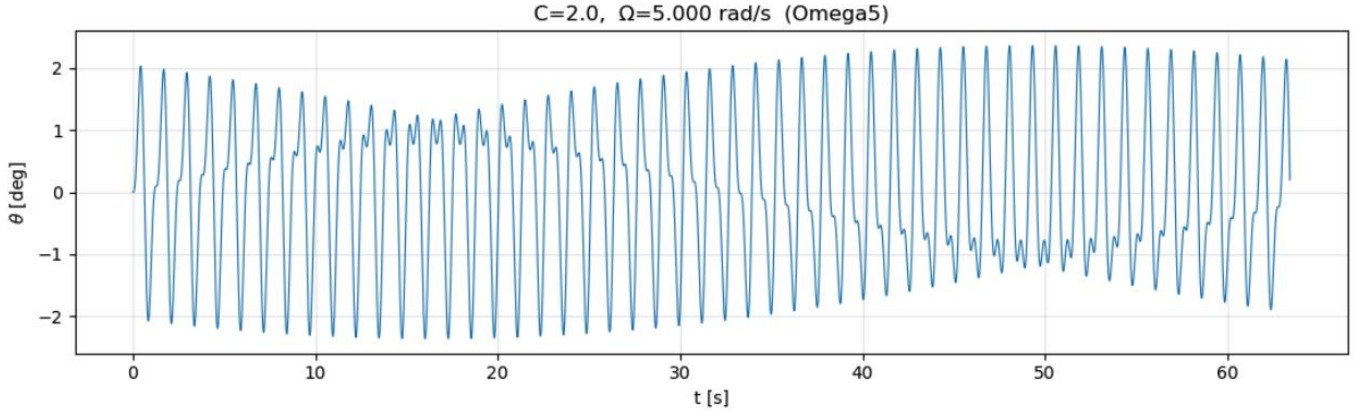
amp = []
for Om in Omegas:
    t_s, x_s = rk4_integrate(make_f_driven(C, Om), 0.0, x0, t_end, h)
    # amplitude in the second half, after transients
    half = t_s >= 0.5 * t_end
    amp.append(0.5 * (np.max(x_s[half, 0]) - np.min(x_s[half, 0])))
amp = np.array(amp)

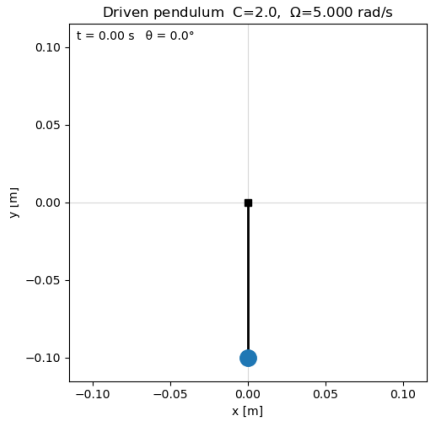
i_max = int(np.argmax(amp))
print(f"Part 4: peak response at Ω ≈ {Omegas[i_max]:.3f} rad/s  "
      f"(ω0 = {OMEGA0:.3f} rad/s),  peak-to-peak/2 = {amp[i_max]:.4f} rad "
      f"= {np.rad2deg(amp[i_max]):.2f} deg")

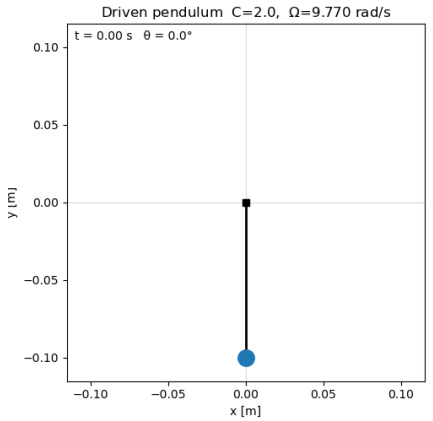
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(Omegas, np.rad2deg(amp), "o-", ms=4)
ax.axvline(OMEGA0, color="k", ls="--", label=rf"$\omega_0=\sqrt{{g/\ell}}={OMEGA0:.2f}$")
ax.set_xlabel(r"driving $\Omega$ [rad/s]")
ax.set_ylabel(r"late-time amplitude [deg]")
ax.set_title(rf"Resonance scan at fixed C = {C}")
ax.legend()
ax.grid(True, alpha=0.3)
fig.tight_layout()
plt.show()
```

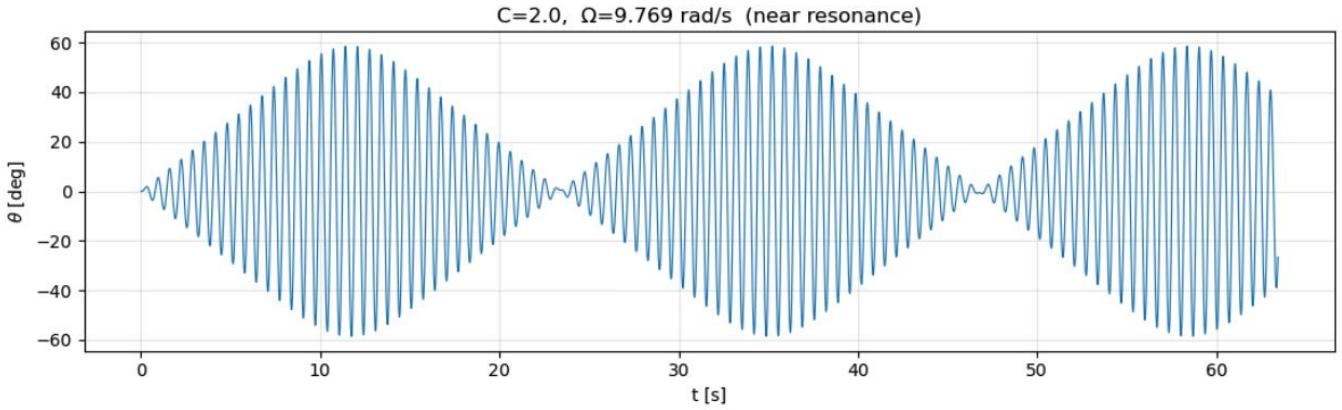
2nd-order ODE: nonlinear pendulum

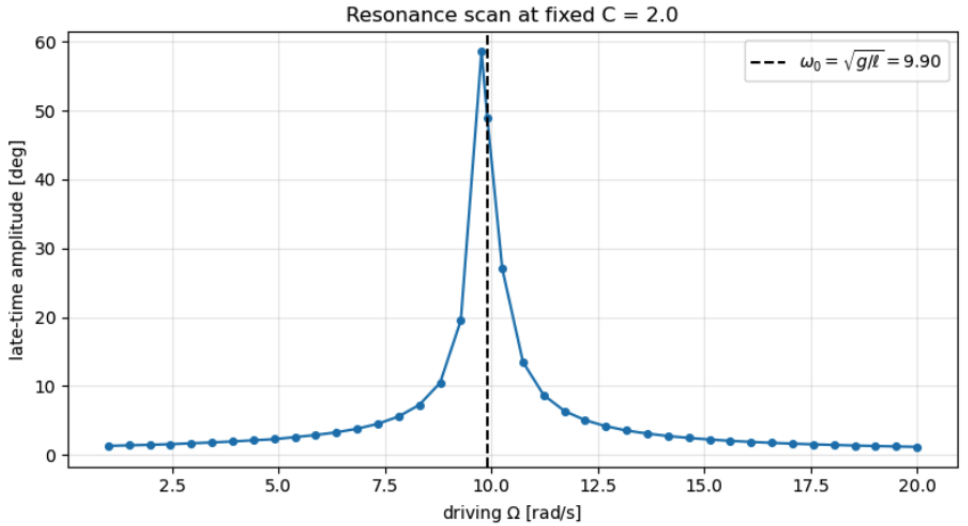
When introducing driving, resonance occurs near ω_0











Adaptive time step

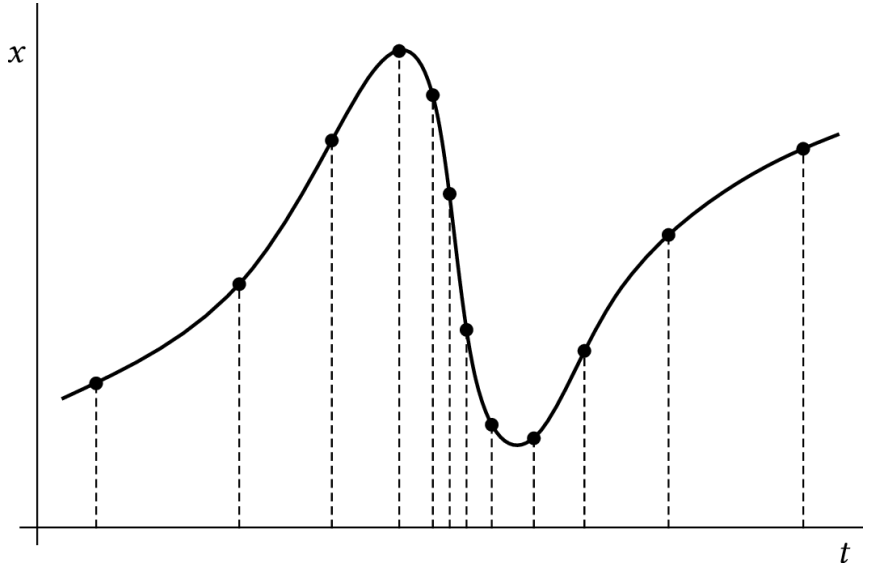
Up to now we solved 1st-order fixing the timestep *h*

The choice of the time step is important to reach the desired accuracy/performance.

- *h* too large: the desired accuracy not reached
- *h* too small: we waste computing resources on unnecessary iterations

Local truncation error itself is a function of time depending on the behavior of f(x,t): when we solve *dx/dt=f(x,t)* there will be

- regions where the function varies slowly, and we could use less and more spaced points
- regions with a more rapid variation which need more points to be accurately computed.



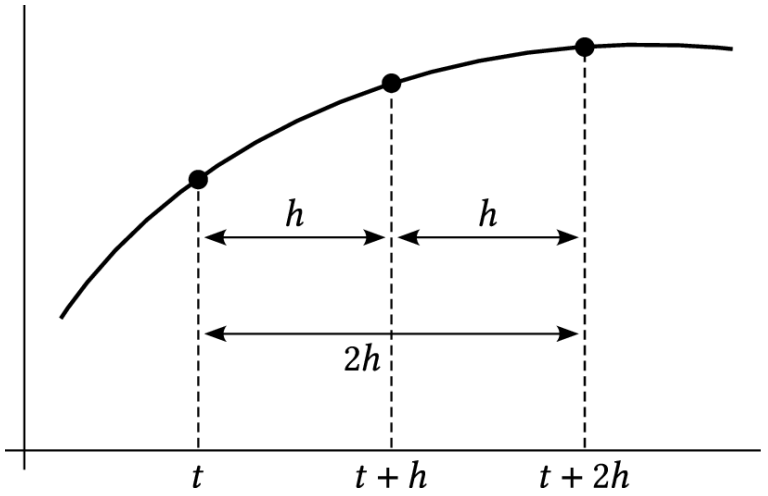
Adaptive time step: make a local error estimate and adjust *h* to correspond to the desired accuracy

Adaptive time step

Basic idea: vary step size *h* so that error introduced per unit interval in *t* is roughly constant

How the approach works for 4th-order Runge-Kutta

- Choose an initial (typically small) *h*
- Starting from *x(t)*, compute two steps of size *h*: *x(t+h), x(t+2h)*
- Start from *x(t),* and compute one step of size *2h*: *x(t+2h)*
- Compare the two estimates to compute the error on the calculation



Let's see now how to compare the two estimates

Adaptive time step in RK4 using double step

Recall that the error for one RK4 time step h is of order ch^5.

Let's call $x(t+2h)$ the true value, x_1 the estimate with step h , x_2 the estimate with step $2h$

With two steps of size h, the error estimate is $2ch^5$:

$$x(t+2h) = x_1 + 2ch^5$$

With one step of size $2h$, the error estimate will be $c(2h)^5 = 32ch^5$:

$$x(t+2h) = x_2 + 32ch^5$$

Equating the two expressions

$$x_1 + 2ch^5 = x_2 + 32ch^5 \longrightarrow \delta = ch^5 = \frac{1}{30}(x_1 - x_2)$$

δ is the **per-step error**

Now if I have a new generic step h' the error will be δ':

$$\delta' = ch'^5 = ch^5\left(\frac{h'}{h}\right)^5 = \frac{1}{30}(x_1 - x_2)\left(\frac{h'}{h}\right)^5$$

Adaptive time step in RK4 using double step

Finally, let's define a target accuracy Δ per unit time.

Suppose now h' is the step with the target accuracy, let's see what it's value should be

$$|\delta'| = h'\Delta \longrightarrow \frac{1}{30}|x_1 - x_2|\left(\frac{h'}{h}\right)^5 = h'\Delta \longrightarrow$$

$$h' = h\left(\frac{30h\Delta}{|x_1 - x_2|}\right)^{1/4} = h\rho^{1/4} \qquad \text{with} \qquad \rho = \frac{30h\Delta}{|x_1 - x_2|}$$

ρ is the ratio between target accuracy for the step *h* and the actual one

Let's go back to the method

Adaptive time step

How the approach works for 4th-order Runge-Kutta

- Choose an initial (typically small) *h*
- Starting from *x(t)*, compute two steps of size *h*: *x(t+h), x(t+2h)*
- Start from *x(t),* and compute one step of size *2h*: *x(t+2h)*
- Compare the two estimates x_1 and x_2 at *t+2h,* by computing the ratio between target accuracy and actual one:

$$\rho = \frac{30h\Delta}{|x_1 - x_2|}$$

  - If $\rho > 1$ the actual accuracy is good and we are using a timestep *h* which is too [redacted]. Keep the result x_1 obtained at *t+2h,* but continue simulation using the step $h' = h\rho^{1/4}$

SMALL

  - If $\rho < 1$ the actual accuracy is poor. Repeat calculation from initial point x(t) using a step $h' = h\rho^{1/4}$

Adaptive time step

Note that:

- we never use x_2, as it is always a poorer estimate than x_1. It is only useful for the error estimation
- we never use an estimate at *x(t+2h)* when $\rho < 1$: even if the deviation is small, the errors are cumulative, and you can end with a large error in the end
- What if you end with x_1 and x_2 very similar? This blows the value of ρ, making *h'* erroneously very large. To avoid it, place an upper limit to how much *h* can increase. A rule of thumb is no more than a factor 2.
- Even if adaptive method involves more computational efforts, it usually pays off in the end.

Adaptive time step

It can be easily extended to **simultaneous ODEs**.

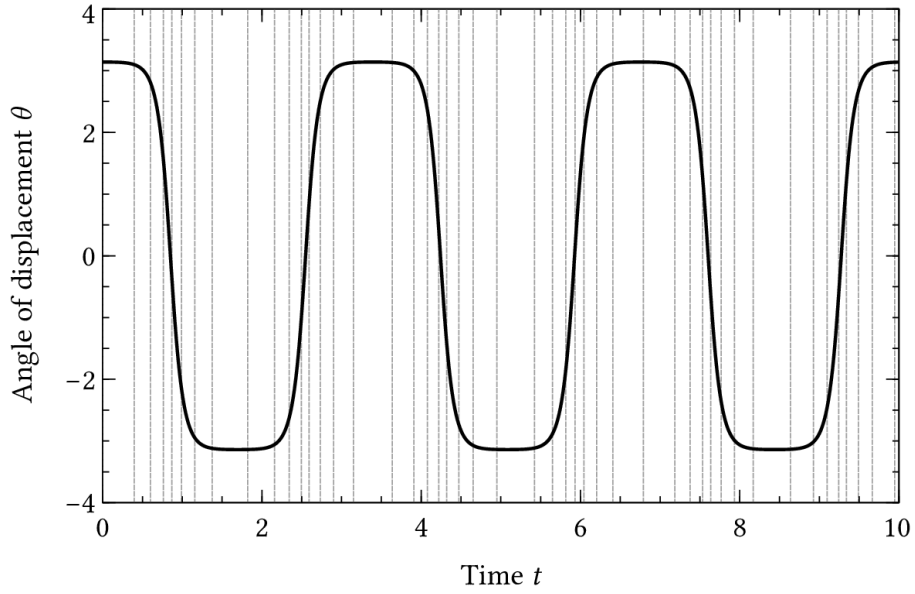
Compute the error for each variable, leading to the same formula:

$\delta_x = \frac{1}{30}(x_1 - x_2), \delta_y = \frac{1}{30}(x_1 - x_2), \ldots$ and then you combine them to choose *h'*.

If it is a single 2nd-order ODE, the second variable does not matter in the error so we can use $\rho = \frac{30h\Delta}{\delta_x}$

(consider e.g. the pendulum, the accuracy can be monitored only looking at θ, given that the error effect through ω is already included)

If (x,y,..) is an Euclidean point, we can use $\rho = \frac{30h\Delta}{\sqrt{\delta_x^2+\delta_y^2+\cdots}}$



Leapfrog method

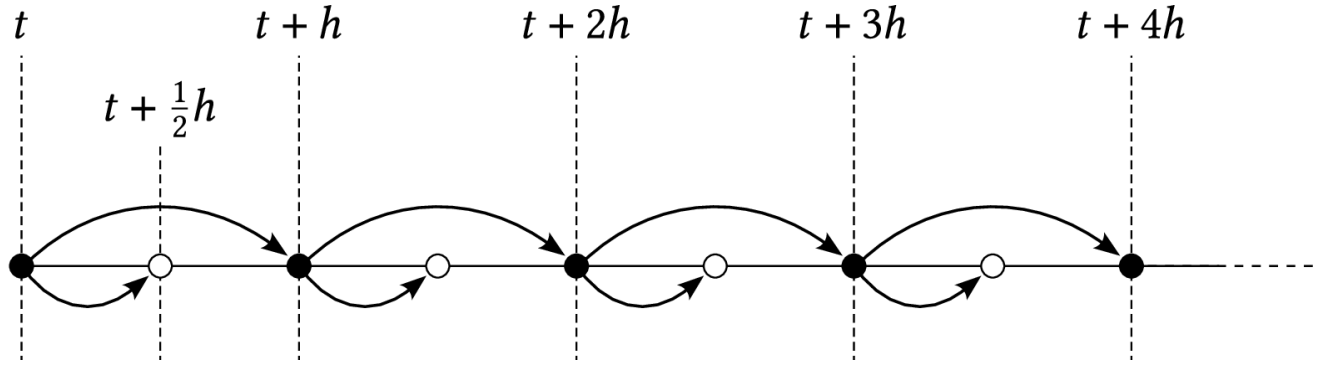
Runge-Kutta is robust and reliable, but other methods have useful properties worth applying in certain contexts.

Start with a 1st-order ODE: $$\frac{dx}{dt} = f(x,t)$$

Remember that for the 2nd-order Runge-Kutta we estimated x(t+h) using the slope at midpoint $f\left(x\left(t+\frac{1}{2}h\right), t+\frac{1}{2}h\right)$ and compute $x\left(t+\frac{1}{2}h\right)$ using Euler's method:

$$x\left(t+\tfrac{1}{2}h\right) = x(t) + \tfrac{1}{2}h\, f(x,t)$$
$$x(t+h) = x(t) + h\, f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right)$$

Graphical visualization: at each step we compute the midpoint and use it as stepping stone to calculate value at *t+h*



Leapfrog method

Now, instead of starting from $x(t+h)$, doing half-step and then full-step, we can do a full step from $x\left(t+\frac{1}{2}h\right)$ to $x\left(t+\frac{3}{2}h\right)$ using the 2nd formula in RK:

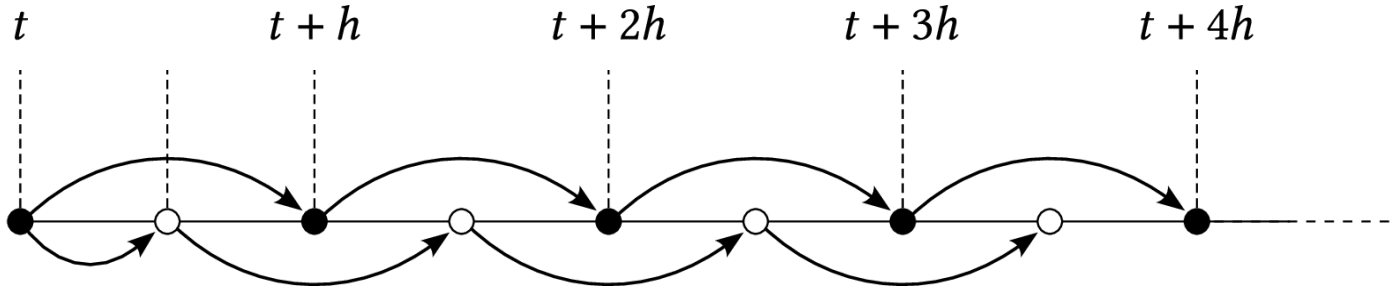
$$x\left(t+\tfrac{3}{2}h\right) = x\left(t+\tfrac{1}{2}h\right) + h\, f\left(x(t+h),\, t+h\right).$$

Here $f(x(t+h), t+h)$ plays the role of the gradient at midpoint between $t+\frac{1}{2}h$ and $t+\frac{3}{2}h$.

We can then use $x\left(t+\frac{3}{2}h\right)$ to compute $x(t+2h)$ using the same formula

$$x(t+2h) = x(t+h) + h\, f\left(x(t+\tfrac{3}{2}h),\, t+\tfrac{3}{2}h\right).$$

After the first 2 steps, which are the same as RK, each midpoint is computed from the previous midpoint. Each step "leaps over" the position of the previously calculated value.



Leapfrog method

Full method:

1) Start from $x(t)$ and $x\left(t+\frac{1}{2}h\right) = x(t) + \frac{1}{2}h\, f(x,t)$ (Euler's half-step, only for first iteration)

2) Repeatedly apply

$$x(t+h) = x(t) + h\, f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right),$$
$$x\left(t+\tfrac{3}{2}h\right) = x\left(t+\tfrac{1}{2}h\right) + h\, f\left(x(t+h),\, t+h\right).$$

Step error is h^3, global error is h^2. Local accuracy is h^2 and global accuracy is h.

Extendable to simultaneous differential equations with vector **x** and vector function **f**(**x**,t)

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h\,\mathbf{f}\left(\mathbf{x}(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right),$$
$$\mathbf{x}\left(t+\tfrac{3}{2}h\right) = \mathbf{x}\left(t+\tfrac{1}{2}h\right) + h\,\mathbf{f}\left(\mathbf{x}(t+h),\, t+h\right).$$

and so to second- or higher-order equations

Leapfrog method: time-reversal symmetry and energy conservation

The method is useful from a physics point of view because it is **time-reversal symmetric.**
Given beginning and end points of time evolution:

$$x(t_1) \text{ and } x\left(t_1 + \frac{1}{2}h\right) \xrightarrow{\text{leapfrog time evolution with } h} x(t_2) \text{ and } x\left(t_2 + \frac{1}{2}h\right)$$

start points — end points

Time-reversal symmetry that if I start from end points and evolve with time interval –h, I will go back to start points:

$$x(t_2) \text{ and } x\left(t_2 + \frac{1}{2}h\right) \xrightarrow{\text{leapfrog time evolution with } -h} x(t_1) \text{ and } x\left(t_1 + \frac{1}{2}h\right)$$

start points — end points

The RK method has not this property

Leapfrog method: time-reversal symmetry and energy conservation

Easy to show. Start from evolution equations

$$x(t+h) = x(t) + h\, f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right),$$
$$x\left(t+\tfrac{3}{2}h\right) = x\left(t+\tfrac{1}{2}h\right) + h\, f\left(x(t+h),\, t+h\right).$$

$\longrightarrow$ $h \to -h$

$$x(t-h) = x(t) - h\, f\left(x(t-\tfrac{1}{2}h),\, t-\tfrac{1}{2}h\right),$$
$$x\left(t-\tfrac{3}{2}h\right) = x\left(t-\tfrac{1}{2}h\right) - h\, f\left(x(t-h),\, t-h\right).$$

$\longrightarrow$ shift time $t \to t + \frac{3}{2}t$

$$x\left(t+\tfrac{1}{2}h\right) = x\left(t+\tfrac{3}{2}h\right) - h\, f\left(x(t+h),\, t+h\right),$$
$$x(t) = x(t+h) - h\, f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right).$$

we obtain the same equations as the starting one, now inverted to obtain the previous points from the following ones. Thus, the system will retrace its steps and recovers the initial values.

Leapfrog method: time-reversal symmetry and energy conservation

Another property is that the algorithm is a **symplectic map**: in a simplified sense the transformation preserves areas (remember Liouville theorem).

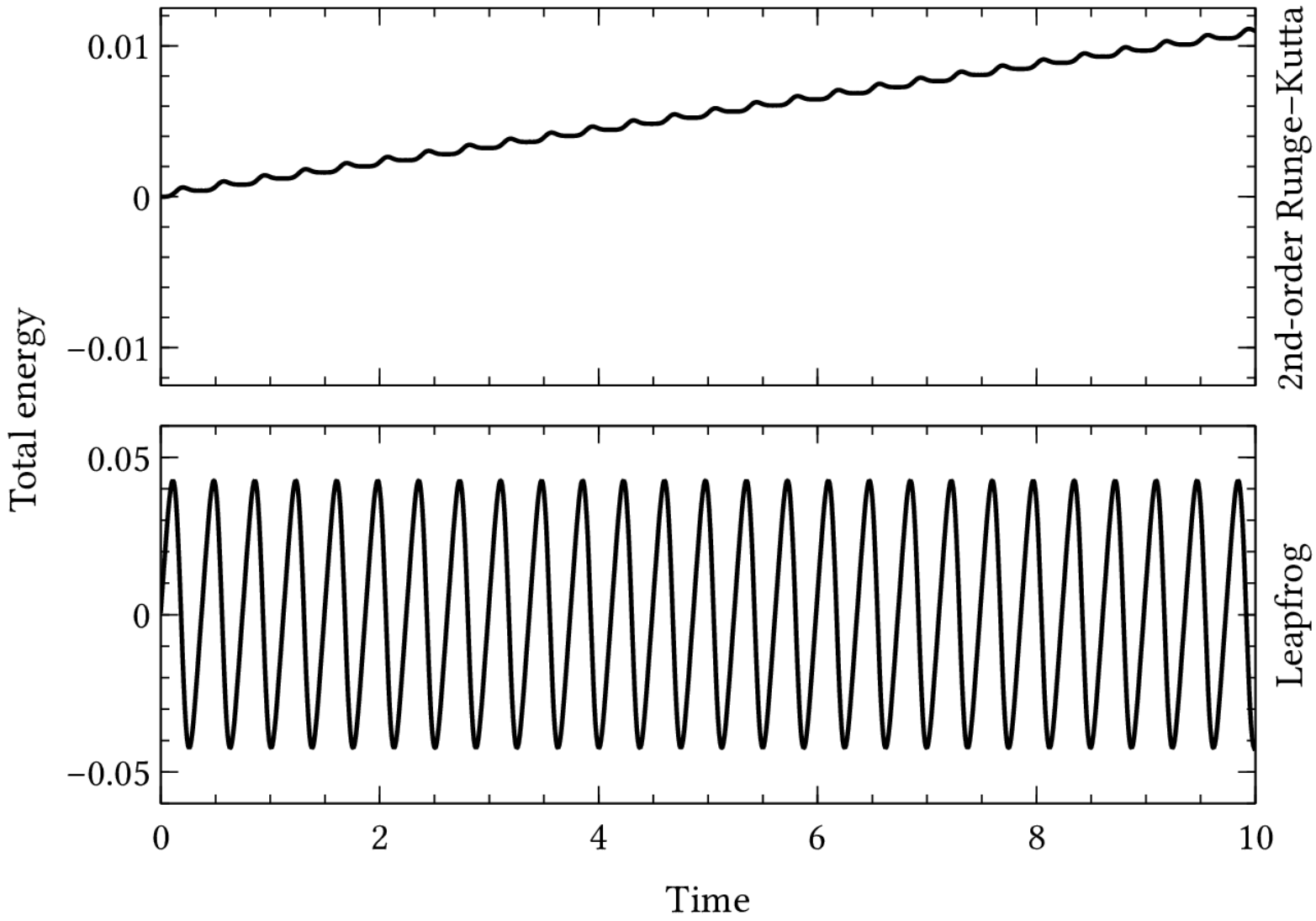
A consequence is that if you are considering a physical system with a Hamiltonian, there exist within the algorithm an associated "shadow" Hamiltonian which remains close to it.

E.g. for the harmonic oscillator with Hamiltonian $H = \frac{p^2}{2m} + \frac{1}{2}m\omega^2 x^2$

the shadow Hamiltonian $\tilde{\mathcal{H}} = \frac{p^2}{2m\left(1 - \omega^2 h^2/4\right)} + \frac{1}{2}m\omega^2 x^2$

$\tilde{\mathcal{H}}$ also indicates that trajectories are ill defined for $h > 2/\omega$: in this case the evolution will not be bounded. In fact, **for the harmonic oscillator the hard limit for the timestep is** $h = 2/\omega$. Below this value, you will observe that H is bounded and slightly fluctuates

Leapfrog method: time-reversal symmetry and energy conservation



Verlet method

Let's work more specifically on a special case, **Newton's equations**, a 2nd-order ODE

$$\frac{\mathrm{d}^2 x}{\mathrm{d}t^2} = f(x,t).$$

This can be converted into coupled 1st-order ODE:

$$\frac{\mathrm{d}x}{\mathrm{d}t} = v, \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = f(x,t).$$

Let's say we apply the leapfrog method to (x,v):

$$x(t+h) = x(t) + h\,v\left(t+\tfrac{1}{2}h\right)$$

$$v(t+h) = v(t) + h\,f\left(x(t+\tfrac{1}{2}h),\, t+\tfrac{1}{2}h\right)$$

$$x\left(t+\tfrac{3}{2}h\right) = x\left(t+\tfrac{1}{2}h\right) + h\,v(t+h)$$

$$v\left(t+\tfrac{3}{2}h\right) = v\left(t+\tfrac{1}{2}h\right) + h\,f\left(x(t+h),\, t+h\right)$$

we don't need these 2 equations to evolve x and v, effectively halving the number of operations

Simplification can be applied only when dx/dt depends on v and not x, and viceversa for dv/dt

Verlet method

$$x(t+h) = x(t) + h\,v\left(t+\tfrac{1}{2}h\right)$$
$$v\left(t+\tfrac{3}{2}h\right) = v\left(t+\tfrac{1}{2}h\right) + h\,f\left(x(t+h),\, t+h\right)$$

x evolves at $t + kh$, with *k* an integer

v evolves at $t + \frac{1}{2}h + kh$

Problem: we don't have access e.g. to the total energy, which needs x and v computed at the same time

We can do an additional half-step only for the velocity. Start from $t + h$ and go back with Euler's method to $-\frac{1}{2}h$

$$v\left(t+\tfrac{1}{2}h\right) = v(t+h) - \tfrac{1}{2}h\,f\left(x(t+h),\, t+h\right) \longrightarrow v(t+h) = v\left(t+\tfrac{1}{2}h\right) + \tfrac{1}{2}h\,f\left(x(t+h),\, t+h\right)$$

this quantity gives *v* at same times as *x* using quantities we know, especially the force *f*

Verlet method

The (velocity) Verlet algorithm looks like this:

- start from an initial condition *x(t), v(t)* and compute the force $f\left(x(t),\, t\right)$
- compute $v\left(t+\frac{1}{2}h\right) = v(t) + \frac{1}{2}h\, f\left(x(t),\, t\right)$

  $x(t+h) = x(t) + h\, v\left(t+\frac{1}{2}h\right)$
- compute the force with new positions $f\left(x(t+h),\, t+h\right)$
- compute $v(t+h) = v\left(t+\frac{1}{2}h\right) + \frac{1}{2}h\, f\left(x(t+h),\, t+h\right)$

Immediately extendable to more than one dimension with $x \to \mathbf{x}$ and $v \to \mathbf{v}$

Boundary value problems

So far we have solved initial value problems. Another possibility is **boundary value problems**

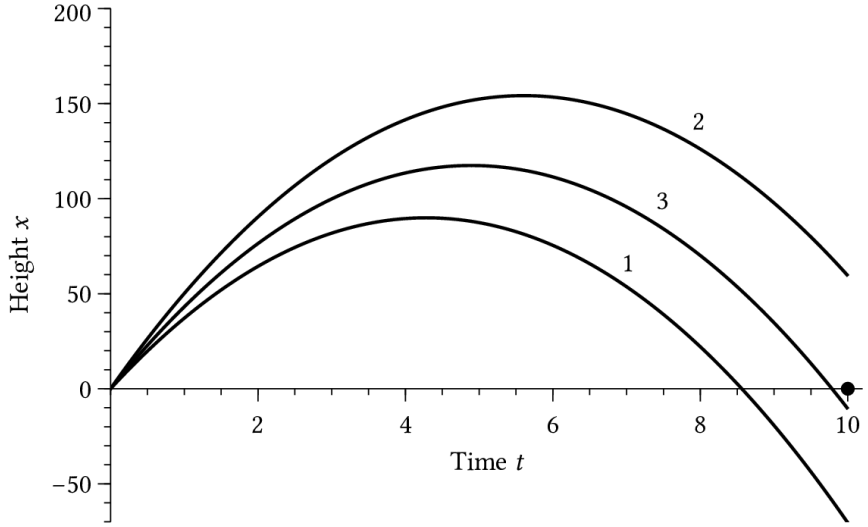
Take for instance the equation governing the height *x* of a ball thrown in the air (ignoring friction)

$$\frac{d^2x}{dt^2} = -g$$

Instead of setting $x(0), v(0)$, we could set the initial positions starting from the ground, and the final position when it touches it again at t_1

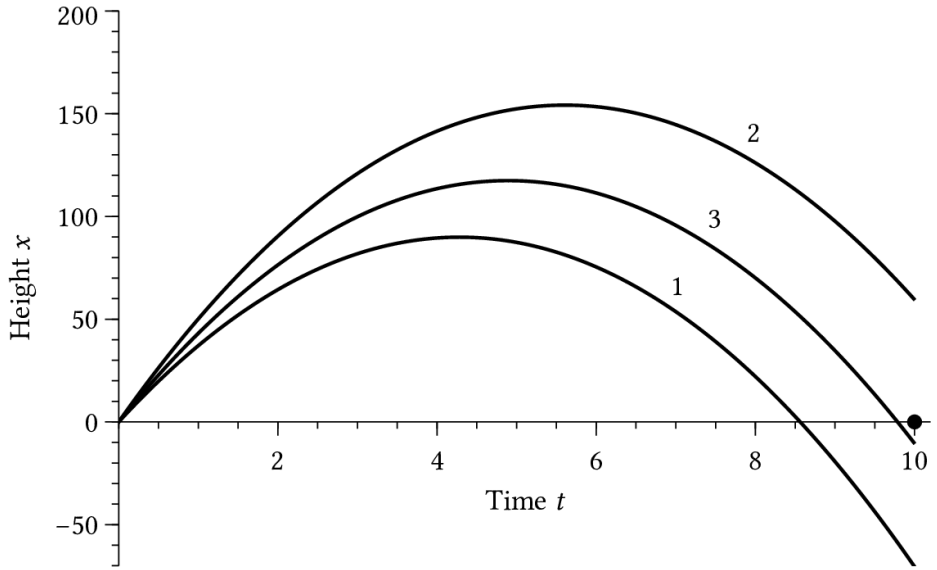
$$x(0) = 0 \text{ and } x(t_1) = 0$$

We want the solution that satisfies these exact conditions. This is a classic artillery fire problem, where we want the exact trajectory that makes a projectile land in a specific point



The shooting method

Shooting method: trial and error method. Vary the initial condition until the evolution gives the correct final condition.



In the artillery problem, we need to find the condition $v(0)$ such that at t_1 $x(t_1) = 0$. The process would look like this:

1) Start by guessing $v(0)$
2) Solve the differential equation up to t_1
3) Check if $x(t_1) = 0$, or if we over- or under-shot. In the latter situations, change $v(0)$ and retry.

How should we modify our guess to converge to the correct value?

In principle there should exist some function x=f(v) that gives $x(t_1)$ as a function of $v(0)$, which we can compute solving each time the differential equation with that initial condition.

The goal is to find f(v)=0, so finding the root of the function, and we know a bunch of methods to solve it.

The shooting method: example

Solve the problem of the throwing ball (two 1st-order ODE) in the case where x=0 at t=10s. Use the 4th-order Runge-Kutta for solving the ODE, and a binary search between v1=0.01 and v2=1000.

$$\frac{d^2x}{dt^2} = -g \longrightarrow \begin{aligned} \frac{dx}{dt} &= v, \\ \frac{dv}{dt} &= -g, \end{aligned}$$

```python
import numpy as np

g = 9.81        # Acceleration due to gravity
a = 0.0         # Initial time
b = 10.0        # Final time
N = 1000        # Number of Runge-Kutta steps
h = (b - a) / N # Size of Runge-Kutta steps
target = 1e-10  # Target accuracy for binary search

def f(r):
    x = r[0]
    y = r[1]
    fx = y
    fy = -g
    return np.array([fx, fy], float)

# Function to solve the equation and calculate the final height
def x(v):
    r = np.array([0.0, v], float)
    for t in np.arange(a, b, h):
        k1 = h * f(r)
        k2 = h * f(r + 0.5 * k1)
        k3 = h * f(r + 0.5 * k2)
        k4 = h * f(r + k3)
        r += (k1 + 2 * k2 + 2 * k3 + k4) / 6
    return r[0]
```

```python
# Main program performs a binary search
v1 = 0.01
v2 = 1000.0

h1 = x(v1)
h2 = x(v2)

while abs(h2 - h1) > target:
    vp = (v1 + v2) / 2
    hp = x(vp)
    if h1 * hp > 0:
        v1 = vp
        h1 = hp
    else:
        v2 = vp
        h2 = hp

v = (v1 + v2) / 2
print("The required initial velocity is", v, "m/s")
```

```
The required initial velocity is 49.04999999999815 m/s
```

Eigenvalue problem and time-independent Schrödinger equation

Sometimes the shooting method will not work by itself. Take for example equations that are linear and homogeneous in the dependent variable, such as the time-independent Schrödinger equation

$$-\frac{\hbar^2}{2m}\frac{d^2\psi}{dx^2} + V(x)\psi(x) = E\psi(x)$$

with $\psi(x)$ the wavefunction, *V(x)* the potential energy and *E* the total energy. Note that **every term depends linearly in** $\psi(x)$.

Let's consider a square well potential of width L and infinitely high walls: $V(x) = \begin{cases} 0 & \text{for } 0 < x < L, \\ \infty & \text{elsewhere.} \end{cases}$

We now want to find the solution $\psi(x)$ that satisfies the boundary conditions $\psi(0) = 0$ and $\psi(L) = 0$ (the probability of finding the particle outside the wall is zero)

Eigenvalue problem and time-independent Schrödinger equation

This appears to be a boundary problem which we could solve with the shooting method.

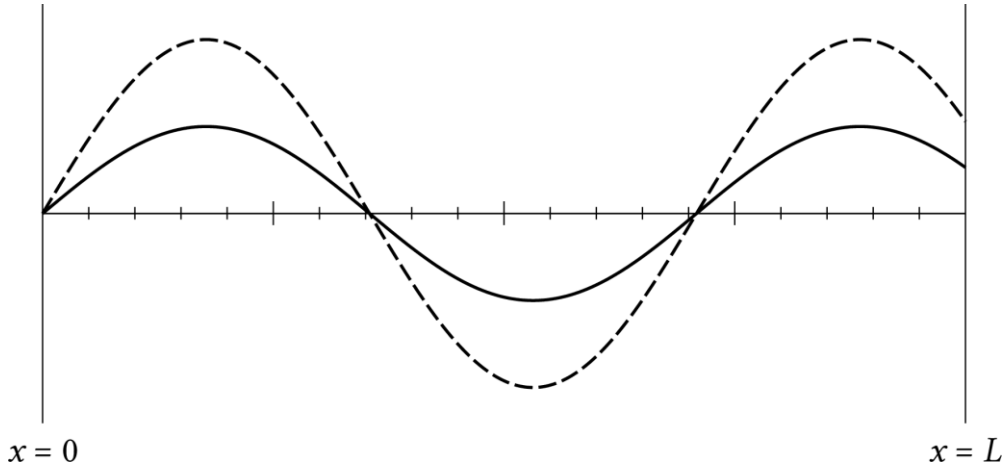
Start by writing the equation in two 1st-order equations $\frac{d\psi}{dx} = \phi, \qquad \frac{d\phi}{dx} = \frac{2m}{\hbar^2}[V(x) - E]\psi$

use the initial condition $\psi(0) = 0$ and guess one condition for the slope of ψ, $\phi(0)$, and integrate using 4th-order RK to find $\psi(x)$. Let's say our guess is wrong, and the solution does not go to $\psi(L) = 0$

We could try changing the condition $\phi(0)$, and try again. But for this problem it does not work:

because Schrödinger equation is linear in ψ , $c\psi(x)$ is also a solution, with initial condition $c\phi(0)$

$$\frac{d}{dx}[c\psi(x)]\bigg|_{x=0} = c\,\phi(0).$$



If $\psi(L) \neq 0$, it will remain as such if I change the slope value.

Eigenvalue problem and time-independent Schrödinger equation

The issue is that there is no solution that satisfies $\psi(0) = 0$ and $\psi(L) = 0$ for any given value of *E*.

There are **only solutions for special values of** E**, the eigenvalues.**

One trick would be then to vary E instead of ϕ. When we find an E for which $\psi(L) = 0$, we have found the allowed energy and the corresponding wavefunction.
Thus we have now a function f that gives E as a function of the value $\psi(L)$, and we want to find the root of it.

The slope value ϕ doesn't matter, since the only effect is a constant multiplication, so we can assign any value.
At the end, after we found ψ, the uncertainty in the multiplying factor is fixed by requiring normalization

$$\int |\psi(x)|^2 \, dx = 1$$

Just compute the integral $\int |\psi(x)|^2 \, dx$ with any numerical method you like, and divide ψ by it.

Example: ground state energy in a square well

Compute the ground state energy of an electron with either of the following two potentials, with L= $0.5292\ \text{Å}$ (Bohr radius) and V_0=100eV.

$$V(x) = \begin{cases} 0 & \text{for } 0 < x < L, \\ \infty & \text{elsewhere.} \end{cases} \qquad V(x) = V_0 \frac{x}{L}\left(\frac{x}{L} - 1\right)$$

Use 4h-order RK to solve the Schrödinger equation

$$\frac{d\psi}{dx} = \phi, \qquad \frac{d\phi}{dx} = C\left[V(x) - E\right]\psi$$

with C=0.2624586 $\text{eV}^{-1}\ \text{Å}^{-2}$

Use the secant method, starting as initial guesses E_1=0 and E_2=1eV.

Plot the wavefunction corresponding to the ground energies found.

Example: ground state energy in a square well

```python
#potentials
def V_infinite(x):
    return 0.0
def V_parabolic(x):
    return V0 * (x / L) * (x / L - 1.0)

# array dpsi/dx dphi/dx
def f(r, x, E, V):
    psi = r[0]
    phi = r[1]
    return np.array([phi, C * (V(x) - E) * psi], float)

#compute wavefunction with 4th order RK
def solve(E, V, wave=False):
    #initial condition psi_0,phi_0
    r = np.array([0.0, 1.0], float)
    x=0
    #save wavefunction
    xs = [x]
    psis = [r[0]]
    for _ in range(N):
        k1 = h * f(r, x, E, V)
        k2 = h * f(r + 0.5 * k1, x + 0.5 * h, E, V)
        k3 = h * f(r + 0.5 * k2, x + 0.5 * h, E, V)
        k4 = h * f(r + k3, x + h, E, V)
        r += (k1 + 2 * k2 + 2 * k3 + k4) / 6
        x += h
        xs.append(x)
        psis.append(r[0])
    if wave:
        return np.array(xs), np.array(psis)
    return r[0]
```

```python
#secant method to find E
def ground_state(V, E1=0.0, E2=1.0, target=1e-3):
    psi2 = solve(E1, V)
    while abs(E1 - E2) > target:
        psi1, psi2 = psi2, solve(E2, V)
        E1, E2 = E2, E2 - psi2 * (E2 - E1) / (psi2 - psi1)
    return E2

# x in Angstrom, E and V in eV
C = 0.26245869297048685
L = 0.5292
N = 1000
h = L / N
V0 = 100.0


#ground energies
E_well = ground_state(V_infinite)
E_par = ground_state(V_parabolic)
print("ground energy square well",E_well)
print("ground energy parabolic well",E_par)

#wave functions
xs, psi = solve(E_well, V_infinite, wave=True)
psi /= np.sqrt(np.trapezoid(psi**2, xs))
plt.plot(xs, psi,label="infinite well")

xs1, psi1 = solve(E_par, V_parabolic, wave=True)
psi1 /= np.sqrt(np.trapezoid(psi**2, xs))
plt.plot(xs1, psi1,label="parabolic well")

plt.xlabel("x (Angstrom)")
plt.ylabel("psi")
plt.legend()
plt.show()
```

```
ground energy square well 134.27622174401935
ground energy parabolic well 112.52995623945586
```

